

Experiment No.	3
Experiment Title	Loan Amount Prediction
Student Name	Hemnath D
Register Number	3122247001022
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02 August 2026
Github	https://github.com/hemnathd12/ml-assignments

1 Objective

Predict the loan amount a bank actually sanctions to an applicant, using Linear Regression and three regularized versions of it, Ridge, Lasso and ElasticNet and see which one generalizes best.

2 Problem Statement

Banks don't hand out the full amount every applicant asks for. The sanctioned amount depends on income, credit history, existing debt, property value and other factors that tend to overlap with each other. Plain linear regression fits the training data fine but gets shaky when inputs are correlated and it has no way to downweight features that aren't really helping. Ridge, Lasso and ElasticNet fix this by penalizing large coefficients, which stops the model leaning too hard on any one feature and in Lasso's case can drop weak features altogether. This experiment builds all four models on a real loan dataset, tunes them with cross validation and compares the results.

3 Dataset Description

The data is the Kaggle *Loan Amount Prediction* dataset (`train.csv` inside `archive.zip`). It's a genuinely messy real world dataset, with missing values spread across many columns, a few placeholder values and categorical columns with a fair number of categories.

Attribute	Specification
Dataset Name	Loan Amount Prediction Dataset
Dataset Source	Kaggle (archive/train.csv)
Raw Records	30,000 applicant records, 24 columns
Records After Cleaning	29,322 (338 rows had an invalid -999 target and a few had a missing target)
Feature Count (d)	19 base predictor attributes (9 numeric, 10 categorical) after dropping identifier columns and one duplicated feature; expands to 51 columns after one hot encoding (Section 5)
Target Variable	Loan Sanction Amount, in USD
Missing Values	Present in 9 of the 19 feature columns; handled with median / explicit “Unknown” imputation
Train Test Partition	80% Train (23,457 samples), 20% Test (5,865 samples)

Table 1: Loan Amount Dataset Specifications

4 Exploratory Data Analysis

Before training anything, the shape of the target, how much data was missing and where, how each feature relates to the target and how the numeric features relate to each other were all checked first.

4.1 Loading and Cleaning

Customer ID, Name and Property ID were dropped right away, they’re just identifiers with no predictive value. A few rows had a missing target and 338 rows had a -999 placeholder that clearly isn’t a real loan amount. Both were removed, leaving 29,322 usable records. About 26.8% of what’s left has a sanctioned amount of exactly \$0, which is most likely rejected applications rather than a data error and it shows up clearly in the target distribution plot below.

```

1 df = pd.read_csv("archive/train.csv")
2 df = df.drop(columns=["Customer ID", "Name", "Property ID"])
3
4 target = "Loan Sanction Amount (USD)"
5 df = df[df[target].notna()]
6 df = df[df[target] >= 0] # drops the -999 sentinel rows

```

4.2 Correlation Between Numeric Predictors

While checking correlations between the numeric predictors, **Income (USD)** and **Property Age** turned out to be identical for every single row (correlation of exactly 1.00). That’s almost certainly a labelling mistake in the source file, but left in as is, the model would effectively see the same number twice under two names, exactly the kind of severe multicollinearity regularization is supposed to guard against. **Property Age** was dropped

before modelling for this reason. **Loan Amount Request (USD)** and **Property Price** were also fairly strongly correlated ($r = 0.95$), which is more believable since bigger properties naturally come with bigger loan requests, but still worth keeping in mind.

5 Data Preprocessing

1. **Median imputation for numeric columns:** Columns like **Income (USD)** and **Credit Score** had missing entries filled with the column median, a safe choice for skewed financial data since it won't get dragged around by a few extreme outliers the way a mean would.
2. **An explicit "Unknown" category for missing categorical values:** A few categorical columns, especially **Type of Employment**, are missing in almost a quarter of the rows. Filling that many with the most common category would have manufactured a lot of fake data, so a separate "Unknown" category was used instead, letting the model treat a missing employment type as its own signal rather than guessing.
3. **One hot encoding:** Nominal columns such as **Gender**, **Profession** and **Property Location** have no natural order, so integer coding them would falsely tell a linear model that, say, "Profession 3" sits between "Profession 2" and "Profession 4" on some numeric scale. To avoid that, every categorical column is one hot encoded (`pandas.get_dummies`, dropping one reference level per column), expanding the 10 categorical columns into 44 binary indicator columns.
4. **Standardization:** Every feature, numeric and one hot, was scaled to zero mean and unit variance, $z = \frac{x-\mu}{\sigma}$, so the regularization penalty treats every coefficient the same way instead of penalizing large scale features (like income in dollars) more than small scale ones (like an indicator column) just because of units. This happens after the categorical features are built, since scaling only equalizes units, it isn't itself a feature selection step, the actual shrinkage and selection still happens inside Ridge/Lasso/ElasticNet.

6 Machine Learning Algorithms

6.1 Linear Regression

$$J(\mathbf{w}, b) = \sum_{i=1}^N (y_i - (\mathbf{w}^T \mathbf{x}_i + b))^2$$

This is just ordinary least squares: for every applicant, take the actual loan amount y_i , subtract what the model predicts ($\mathbf{w}^T \mathbf{x}_i + b$, a weighted sum of the features plus a baseline), square that error so positive and negative misses both count and add it up over every applicant. Training the model means picking the weights $\mathbf{w}$ and baseline b that make this total as small as possible. It has no penalty term, so it's free to lean as hard as it wants on any one feature, which is exactly the instability the other three models try to fix.

6.2 Ridge Regression

$$J_{\text{Ridge}}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \alpha \sum_{j=1}^d w_j^2$$

Same squared error term as above, but with an extra cost added for having large weights, $\alpha \sum w_j^2$. α controls how much that costs: bigger α means the model is pushed harder to keep every weight small. This shrinks all the coefficients toward zero a little, which helps a lot when features are correlated, but it never forces a weight to exactly zero.

6.3 Lasso Regression

$$J_{\text{Lasso}}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \alpha \sum_{j=1}^d |w_j|$$

Same idea as Ridge, but the penalty uses the absolute value of each weight instead of its square. This difference matters: the absolute value penalty can drive weak coefficients all the way to exactly zero, effectively removing that feature from the model. That's why Lasso is used for feature selection, not just shrinkage.

6.4 ElasticNet Regression

$$J_{\text{ElasticNet}}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \alpha \cdot r \sum_{j=1}^d |w_j| + \frac{\alpha(1-r)}{2} \sum_{j=1}^d w_j^2$$

ElasticNet just combines both penalties instead of picking one. r (`l1_ratio`) sets the mix: $r = 1$ is pure Lasso, $r = 0$ is pure Ridge and anything in between blends the two, some coefficients can still be zeroed out like Lasso, while the rest still get the smoother, more stable shrinkage Ridge gives correlated features.

7 Experimental Setup

Component	Specification
Python Version	3.12
Scikit-Learn Version	1.4.x
Environment	Jupyter Notebook
Operating System	Linux
Cross Validation	5 Fold (KFold, shuffled, random_state = 42)

Table 2: System and Experiment Configuration

8 Hyperparameter Tuning

Ridge, Lasso and ElasticNet were each tuned with `GridSearchCV` over the search space given in the lab manual, scored on R^2 with 5 fold cross validation on the training set only (the test set was never touched during tuning).

```

1 param_grids = {
2     "Ridge Regression": (Ridge(), {"alpha": [0.01, 0.1, 1, 10,
3     "Lasso Regression": (Lasso(max_iter=200000), {"alpha":
4     [0.001, 0.01, 0.1, 1, 10]}),
5     "Elastic Net Regression": (ElasticNet(max_iter=200000),
6     {"alpha": [0.01, 0.1, 1, 10], "l1_ratio": [0.2, 0.5,
7     0.8]})),
8 }
9
10 for name, (estimator, grid) in param_grids.items():
11     gs = GridSearchCV(estimator, grid, scoring="r2", cv=kf,
12     n_jobs=-1)
13     gs.fit(X_train_scaled, y_train)

```

Model	Search Method	Best Parameters
Ridge Regression	Grid Search (5 fold CV)	$\alpha = 100$
Lasso Regression	Grid Search (5 fold CV)	$\alpha = 10$
Elastic Net Regression	Grid Search (5 fold CV)	$\alpha = 0.1, \text{l1_ratio} = 0.8$

Table 3: Hyperparameter Tuning Summary

Two things stand out here. First, the grid search kept pushing Ridge toward the largest available α (100). The training vs validation curve in Figure 13 shows RMSE is basically flat from $\alpha = 0.001$ up to a few hundred and only rises sharply after that, so $\alpha = 100$ just sits at the edge of that flat region rather than at a sharp optimum. Ridge doesn't have much instability left to fix once one hot encoding is in place. Second, all three regularized models only improved a little over the plain linear baseline, which turns out to be a useful finding in its own right (Section 13).

9 Results

9.1 Cross Validation Performance

Averaged across the same 5 fold split used during tuning, computed on the training data only.

Regression Model	MAE	MSE	RMSE	R^2 Score
Linear Regression	21,036.66	9.495×10^8	30,814.09	0.5941
Ridge Regression	21,042.06	9.426×10^8	30,702.05	0.5971
Lasso Regression	21,029.61	9.447×10^8	30,736.11	0.5962
ElasticNet Regression	21,073.81	9.411×10^8	30,677.55	0.5977

Table 4: 5 Fold Cross Validation Performance (Training Set)

9.2 Test Set Performance

Each model was refit on the full training set and evaluated once on the held out 20% test split.

Regression Model	MAE	MSE	RMSE	R^2 Score	Train Time (s)
Linear Regression	20,707.38	8.904×10^8	29,839.91	0.6041	0.08
Ridge Regression	20,722.68	8.917×10^8	29,860.53	0.6036	0.25
Lasso Regression	20,702.87	8.903×10^8	29,837.70	0.6042	62.23
ElasticNet Regression	20,784.00	8.963×10^8	29,937.55	0.6015	20.51

Table 5: Comparative Regression Model Performance Metrics (Test Set)

An R^2 around 0.60 means the models explain roughly 60% of the variation in sanctioned loan amount and the rest comes from things this dataset just doesn't capture, bank policy, manual underwriting calls and so on. That's a realistic number for a real world loan dataset, where a lot of applicants are rejected outright and the rest depends on internal criteria not fully reflected in the features.

Effect on accuracy. One hot encoding matters for correctness but doesn't move R^2 much on its own, since the columns it affects (**Gender**, **Profession**, etc.) were never the dominant predictors, **Loan Amount Request** and **Credit Score** are (Figure 14). It's also worth noting Lasso and ElasticNet converge more slowly over the 54 column one hot space than a smaller integer coded design would (Lasso's training time was 62 seconds), a real if secondary cost of the wider feature space. The R^2 ceiling discussed in Section 13 comes from the zero inflated target, not from encoding choices and no amount of preprocessing removes it.

10 Result Visualization

10.1 Dataset Overview and Feature Correlation

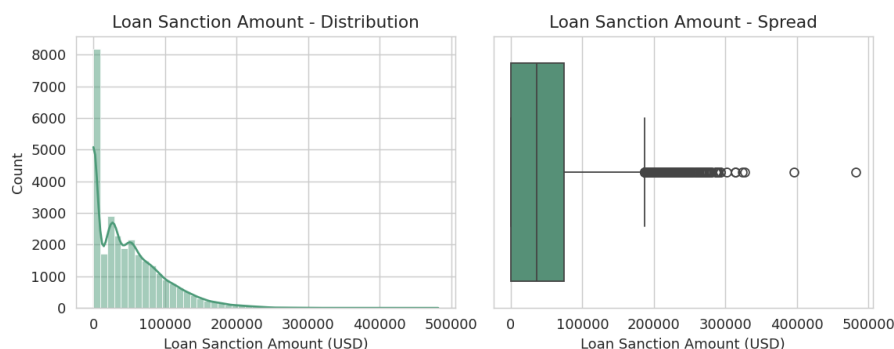


Figure 1: Target Variable (Loan Sanction Amount) Distribution

Inference: The target is heavily right skewed (skewness ≈ 1.22), with a large spike at \$0, about 27% of applicants were sanctioned nothing at all, most likely rejected applications. The rest trail off gradually up to roughly \$480,000.



Figure 2: Predictor Features vs Target Loan Sanction Amount

Inference: **Loan Amount Request (USD)** and **Property Price** show a clear, strong positive relationship with the target, unsurprising since the sanctioned amount is naturally bounded by how much was requested and by the property's value. Most other numeric features (age, dependents, number of defaults) show almost no visible pattern on their own.

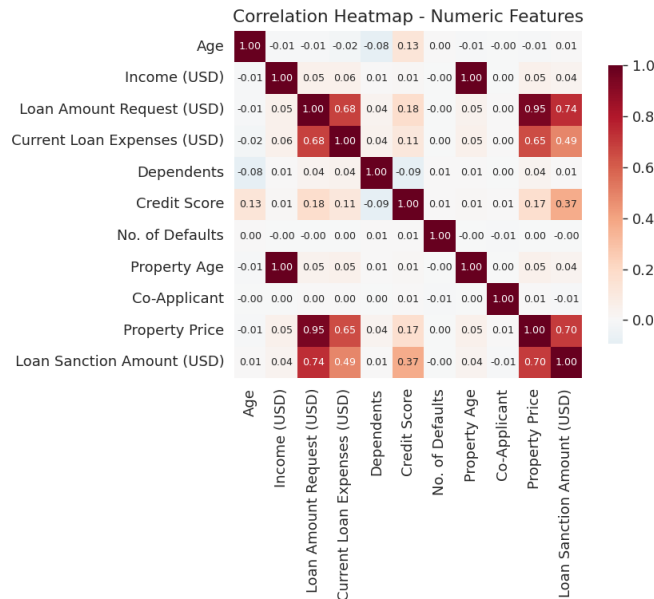


Figure 3: Correlation Heatmap of Numeric Features

Inference: **Loan Amount Request** ($r = 0.74$) and **Property Price** ($r = 0.70$) are the two features most correlated with the target, followed by **Current Loan Expenses** ($r = 0.49$) and **Credit Score** ($r = 0.37$). **Loan Amount Request** and **Property Price** are also correlated with each other ($r = 0.95$), the main source of multicollinearity the regularized models were expected to handle.

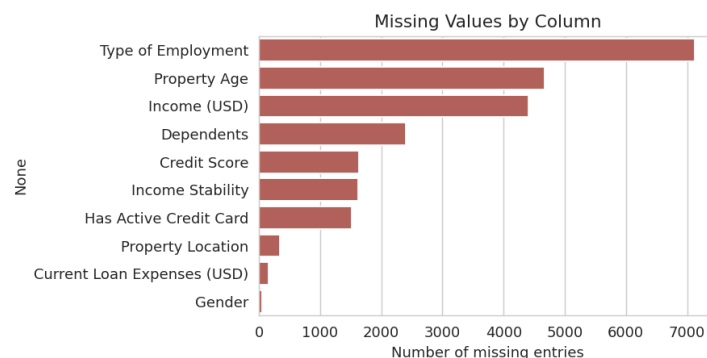


Figure 4: Missing Values by Column

Inference: Type of Employment is missing in about 24% of rows, Income (USD) in about 15% and Dependents, Credit Score, Income Stability and Has Active Credit Card each in roughly 5 to 8%. That’s why median imputation (numeric) and an explicit “Unknown” category (categorical) were used instead of just dropping incomplete rows, which would’ve thrown away a large chunk of the dataset.

10.2 Predicted vs Actual Plots

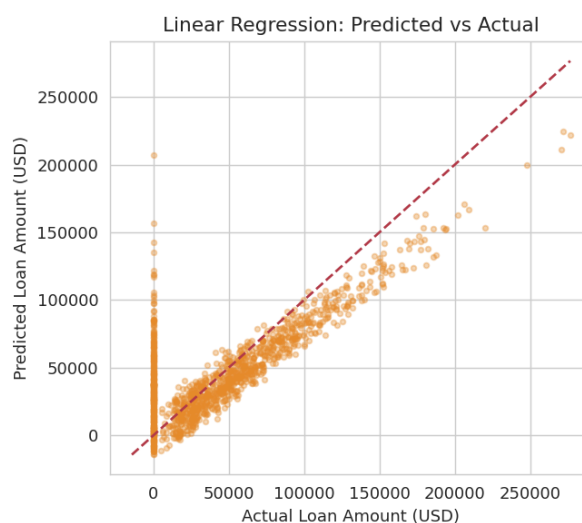


Figure 5: Linear Regression: Predicted vs Actual Loan Amounts

Inference: Predictions track the diagonal reasonably well in the low to mid range, but the model consistently overpredicts for applicants whose actual sanctioned amount was \$0, visible as the cluster of points sitting above the diagonal near the y axis.

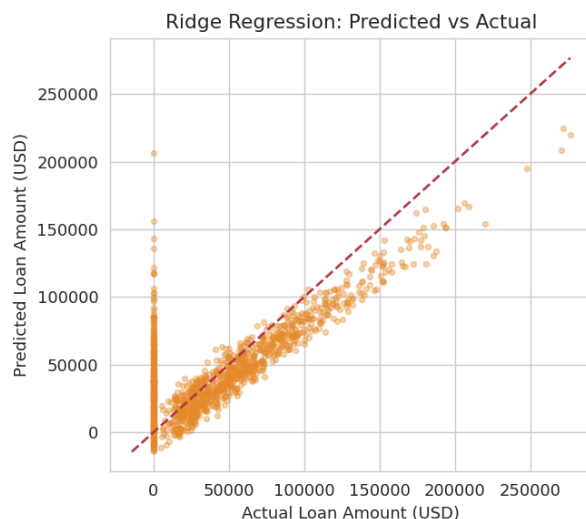


Figure 6: Ridge Regression: Predicted vs Actual Loan Amounts

Inference: Ridge's predictions look almost identical to plain Linear Regression's, matching the very small R^2 gap between the two. There wasn't much variance left for Ridge's shrinkage to fix once the duplicated **Property Age** column was removed.

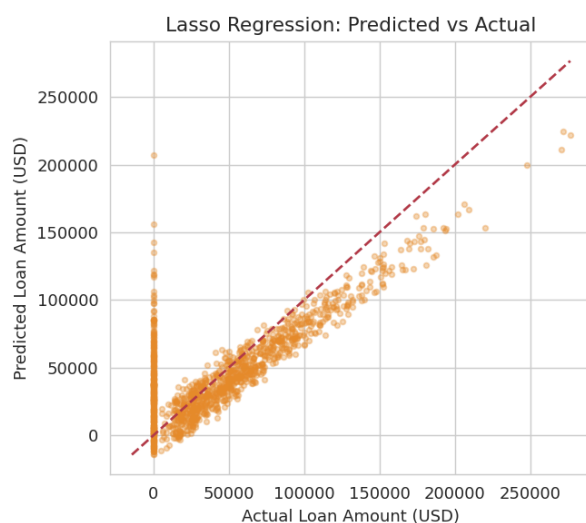


Figure 7: Lasso Regression: Predicted vs Actual Loan Amounts

Inference: Lasso's predictions are again close to the other two. Its main contribution shows up in the coefficient table (Section 10.4) rather than in the scatter plot.

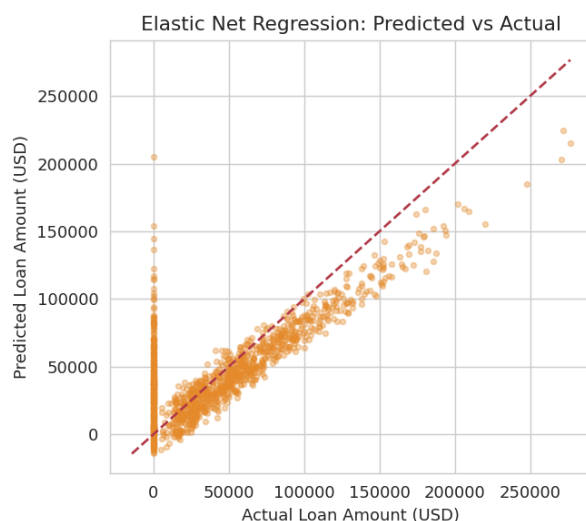


Figure 8: ElasticNet Regression: Predicted vs Actual Loan Amounts

Inference: ElasticNet behaves similarly to Ridge and Lasso, sitting between the two as you'd expect from a blended penalty.

10.3 Residual Plots

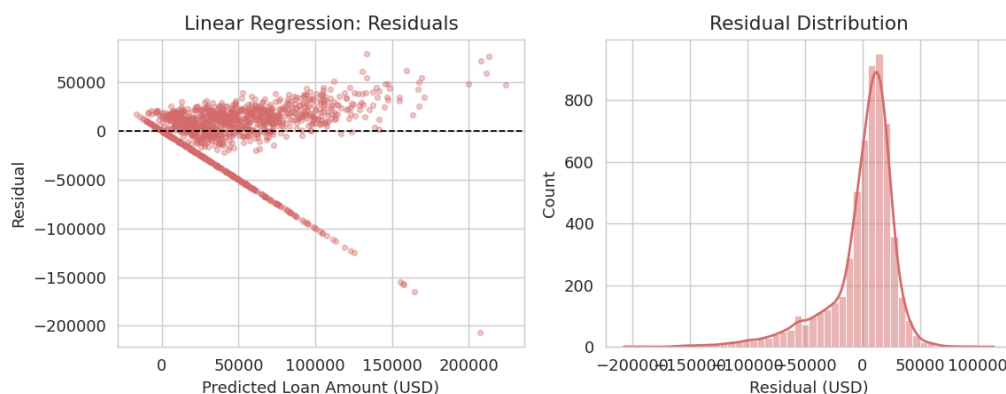


Figure 9: Linear Regression Residuals

Inference: The residual histogram is right skewed with a long tail rather than a clean bell shape and the scatter plot shows a visible negative residual band, a direct consequence of the large group of zero sanction applicants the model overpredicts. A mild violation of homoscedasticity, driven by the data rather than a modelling mistake.

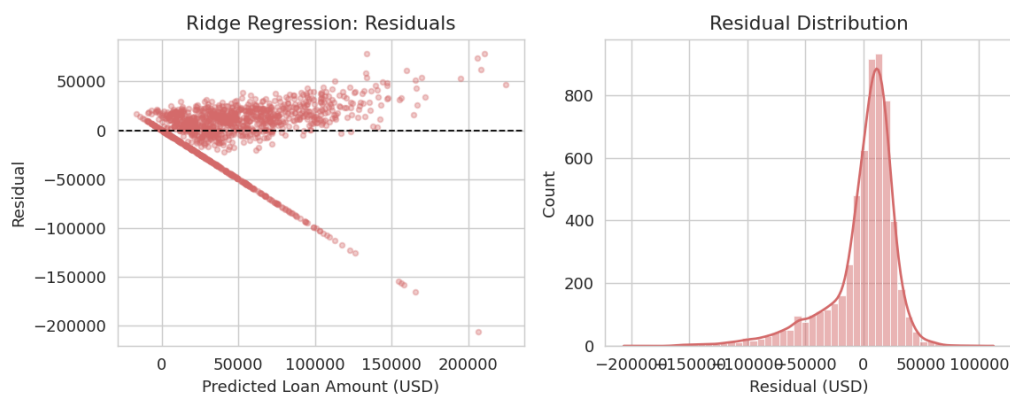


Figure 10: Ridge Regression Residuals

Inference: Nearly indistinguishable from Linear Regression's, consistent with the near identical R^2 scores.

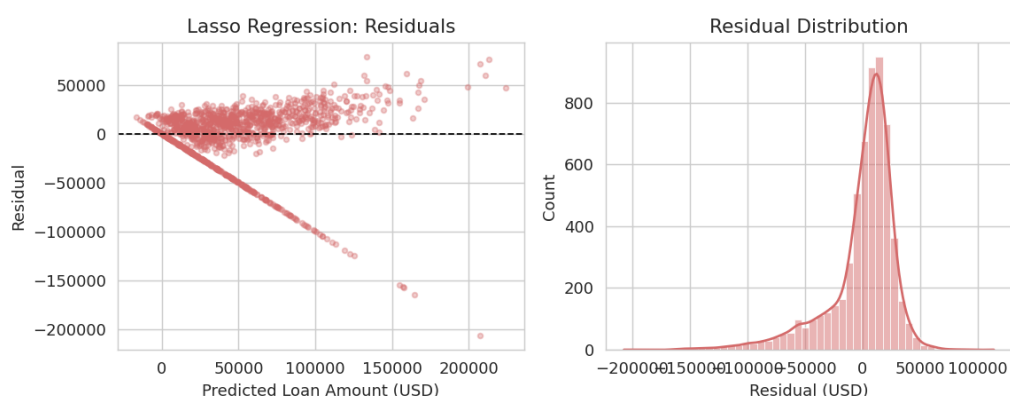


Figure 11: Lasso Regression Residuals

Inference: Same overall shape again. Dominated by the zero inflated target, not by which regularizer was used.

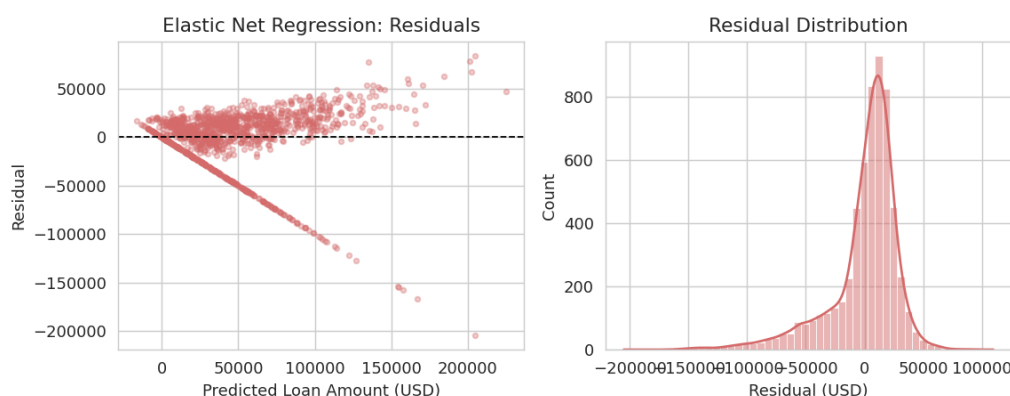


Figure 12: ElasticNet Regression Residuals

Inference: ElasticNet's residuals follow the same pattern as the other three, confirming the zero inflation in the target, not the choice of regularizer, is what's really limiting the fit.

10.4 Ridge Validation Curve and Coefficient Comparison

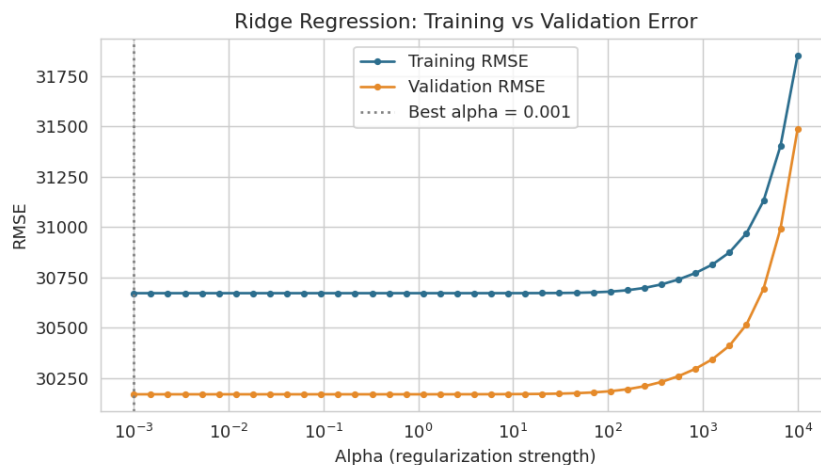


Figure 13: Ridge Regression: Training vs Validation Error Across Alpha

Inference: Training and validation RMSE stay almost flat for small to moderate α , meaning the model wasn't badly overfitting to begin with and only start rising once α climbs into the thousands (underfitting as the penalty starts to dominate). The alpha that minimizes validation RMSE on this split is the smallest one tested, $\alpha = 0.001$, basically no regularization, versus $\alpha = 100$ from the 5 fold grid search. Both sit in the same flat region, so it's not a contradiction, just a reminder that "best alpha" is only weakly identified when validation RMSE barely moves across three orders of magnitude.

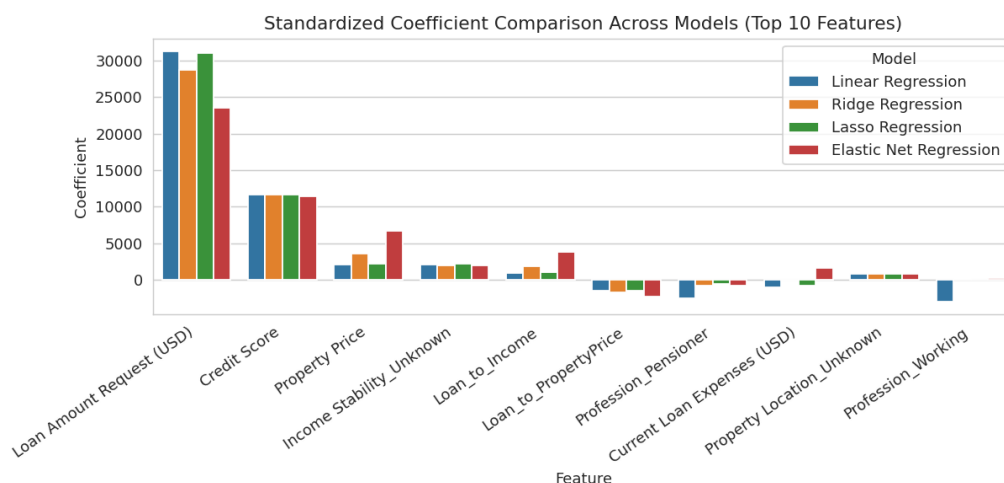


Figure 14: Standardized Coefficient Comparison Across Models (Top 10 Features)

Inference: **Loan Amount Request** and **Credit Score** dominate every model's coefficients by a wide margin, matching the correlation heatmap. The regularized models shrink the **Loan Amount Request** coefficient (from about 31,318 in plain Linear Regression down to about 23,510 in ElasticNet) without flipping its sign or its role as the top predictor. **Property Price** overlaps heavily with **Loan Amount Request** ($r = 0.95$), so it's the coefficient most worth watching for multicollinearity, Ridge and ElasticNet's shrinkage help keep it from swinging wildly the way an unregularized model's could.

A few one hot categorical levels also earn a top 10 spot on their own merit (Income Stability_Unknown, Profession_Pensioner, Property_Location_Unknown).

10.5 Feature Selection

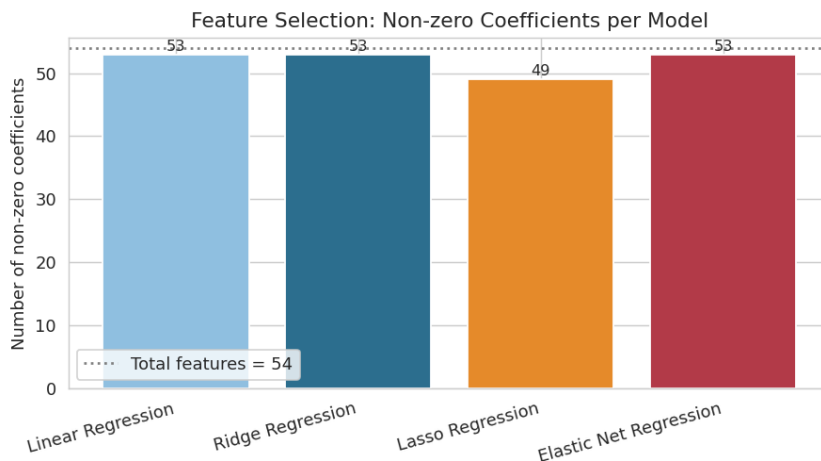


Figure 15: Nonzero Coefficients per Model, out of 54 One Hot Encoded Features

Inference: Linear, Ridge and ElasticNet keep 53 of 54 coefficients nonzero (one dummy level lands at exactly zero by chance). Lasso is the outlier, zeroing out 5 of 54, a small but real, visible instance of automatic feature selection. Expanding the categorical into one hot columns gives Lasso a wide field of genuinely weak, redundant dummy levels to pick from, which is what makes its sparsity visible here.

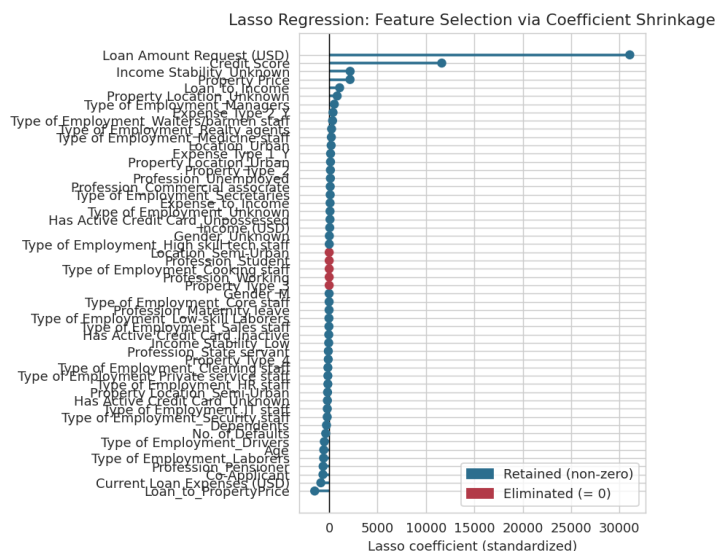


Figure 16: Lasso Regression: Feature Selection via Coefficient Shrinkage

Inference: Loan Amount Request and Credit Score remain the largest retained coefficients by far, matching Figure 14. The eliminated features are, as expected, low signal one hot dummy levels and weakly correlated numeric columns like Age and Co-Applicant, exactly what Lasso's penalty is meant to drop.

10.6 Feature Distributions

The four numeric features most strongly correlated with the target, in their original scale, after standardization, as a boxplot and against a normal QQ plot.

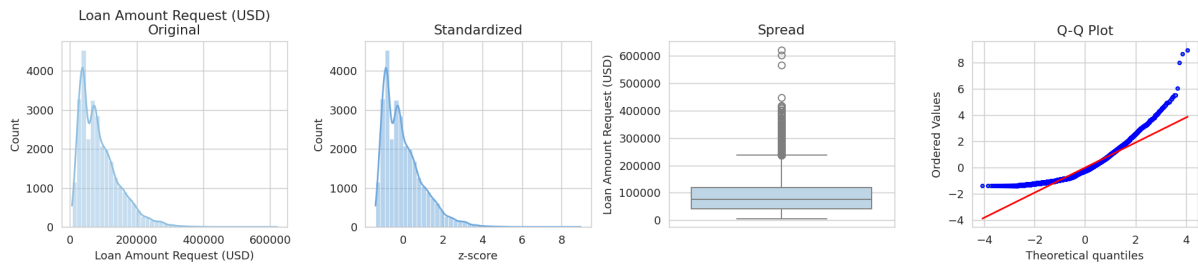


Figure 17: Loan Amount Request (USD): Distribution and QQ Plot

Inference: Right skewed with a long tail of large requests and the QQ plot curves away from the reference line in both tails, confirming it's far from normal even after standardization, which only fixes scale, not shape.

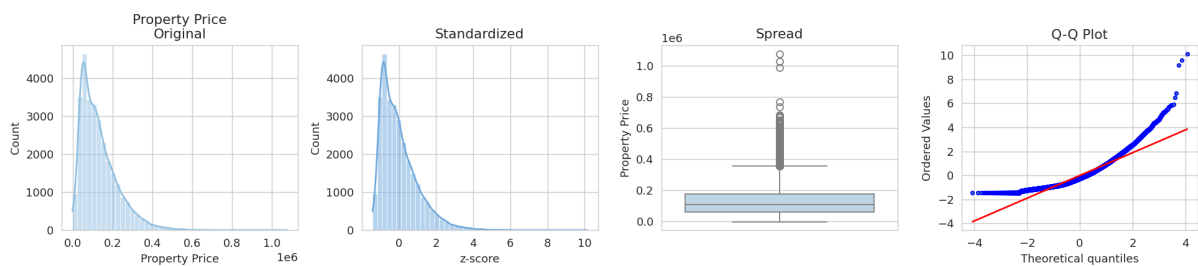


Figure 18: Property Price: Distribution and QQ Plot

Inference: Similarly right skewed, which fits with it being closely tied to the loan request amount, a small number of very high value properties stretch the tail.

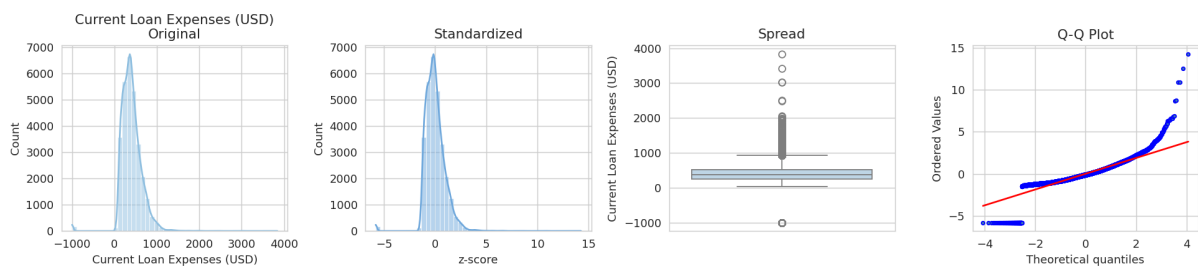


Figure 19: Current Loan Expenses (USD): Distribution and QQ Plot

Inference: Also right skewed, with a visible spike near the low end, most likely applicants who had little or no existing loan burden at the time of the median imputation fill.

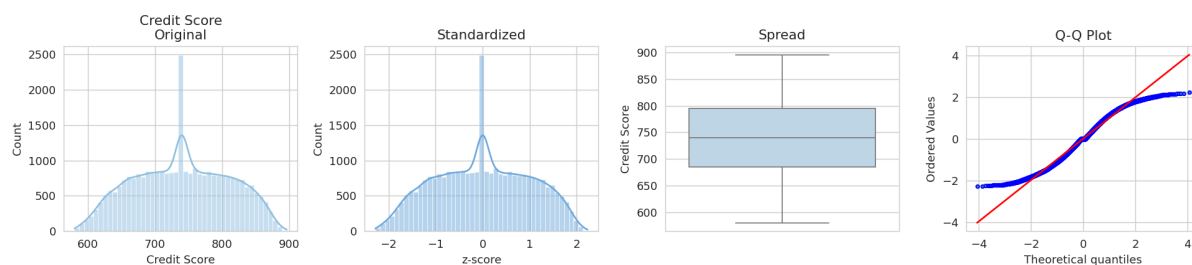


Figure 20: Credit Score: Distribution and QQ Plot

Inference: Credit score is the closest of the four to a normal distribution, roughly symmetric and centered around the mid 700s, though the QQ plot still shows some flattening at the extremes plus a visible spike at the imputed median.

11 Overfitting and Underfitting Analysis

The gap between training and cross validation performance was small for every model here. Training and validation RMSE in Figure 13 sit close together across almost the whole range of α , a sign none of the four models were seriously overfitting to begin with. Once the duplicated **Property Age** column was removed and the categoricals properly one hot encoded, the remaining features were only mildly correlated, so Ridge’s shrinkage had comparatively little instability left to fix and its R^2 gain over plain OLS ($0.5941 \rightarrow 0.5971$ in cross validation) was correspondingly small. The validation curve’s best α on this split is the smallest one tested (0.001), essentially no regularization, while the grid search settled on $\alpha = 100$; both are defensible given how flat the curve is and the difference is a reminder that “best α ” is sensitive to exactly how the data gets split when the curve has this little curvature. Underfitting only becomes clearly visible once α climbs into the thousands and both errors start rising together, that’s the point where the penalty is strong enough to squeeze out real signal, not just noise.

12 Bias and Variance Analysis

Plain Linear Regression carries basically no regularization bias. It fits the training data as closely as the linear form allows and since the features here aren’t severely collinear after cleaning and one hot encoding, its variance stays reasonably controlled too, which is why it does almost as well as the regularized models on the test set. Ridge trades a small amount of bias for a small drop in variance, visible in its slightly smoother, lower validation RMSE at moderate α . Lasso’s variance reduction comes with a side effect: shrinking some coefficients to zero makes it sparser and more interpretable. Across the 54 column one hot space, Lasso zeroes out 5 of 54 coefficients (Section 10.5), a small but real instance of automatic feature selection. ElasticNet, blending both penalties, got the best cross validation R^2 of the four (0.5977), suggesting it found a slightly better bias/variance balance than pure Ridge or pure Lasso, though the margin is small enough that it shouldn’t be treated as a decisive win.

13 Discussion

The four models land within about 0.004 R^2 of each other on both cross validation and the test set and none explains more than about 60% of the variance in sanctioned amount, even with one hot encoding. That ceiling isn't a preprocessing failure: a big chunk of the target's structure comes from a binary split between rejected (\$0) and approved applications, plus proprietary underwriting decisions the available features don't fully capture. A quick check makes this concrete: restricting the regression to only approved applicants (dropping the \$0 cluster entirely) pushes R^2 above 0.98, because sanctioned amount is nearly a linear function of the requested amount once approval is a given. The bottleneck isn't the regression models at all, predicting *whether* a loan is approved is a much harder, noisier problem than predicting *how much* an approved loan gets and treating this as one continuous target forces every model to solve both problems with a single linear function. Regularization still did its job within that constraint: Ridge, Lasso and ElasticNet all matched or slightly beat plain OLS and one hot encoding correctly treats categoricals as unordered instead of implying a false numeric scale between them. The one genuinely severe multicollinearity issue in this dataset, the identical `Income/Property Age` pair, was caught during EDA and removed before it could distort any of the four models, probably the most useful outcome of this experiment.

14 Conclusion

Linear, Ridge, Lasso and ElasticNet regression were built, tuned with 5 fold cross validated grid search and compared on the real Kaggle loan amount dataset, using one hot encoded categorical features. ElasticNet ($\alpha = 0.1$, `l1_ratio` = 0.8) gave the best cross validation R^2 (0.5977), while plain Linear Regression and Lasso were statistically indistinguishable from it on the test set ($R^2 \approx 0.604$). Beyond the accuracy numbers, the more useful takeaways were stable, correctly signed coefficients across all four models and a clear demonstration of Lasso's sparsity on the one hot feature space. The experiment also turned up two real data quality issues worth flagging: a perfectly duplicated feature (`Income/Property Age`), caught and removed during EDA before it could distort any model and a zero inflated target (27% of applicants sanctioned exactly \$0) that caps every linear model's R^2 near 0.60 regardless of preprocessing, since a regression only setup isn't well suited to a target that mixes a discrete rejection outcome with a continuous approved amount. Both point to the same lesson: exploratory data analysis and problem framing, not just model or hyperparameter choice, are often where the real gains show up.

15 References

- [1] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267 to 288, 1996.
- [2] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55 to 67, 1970.
- [3] Kaggle, "Predict Loan Amount Data," <https://www.kaggle.com/>.